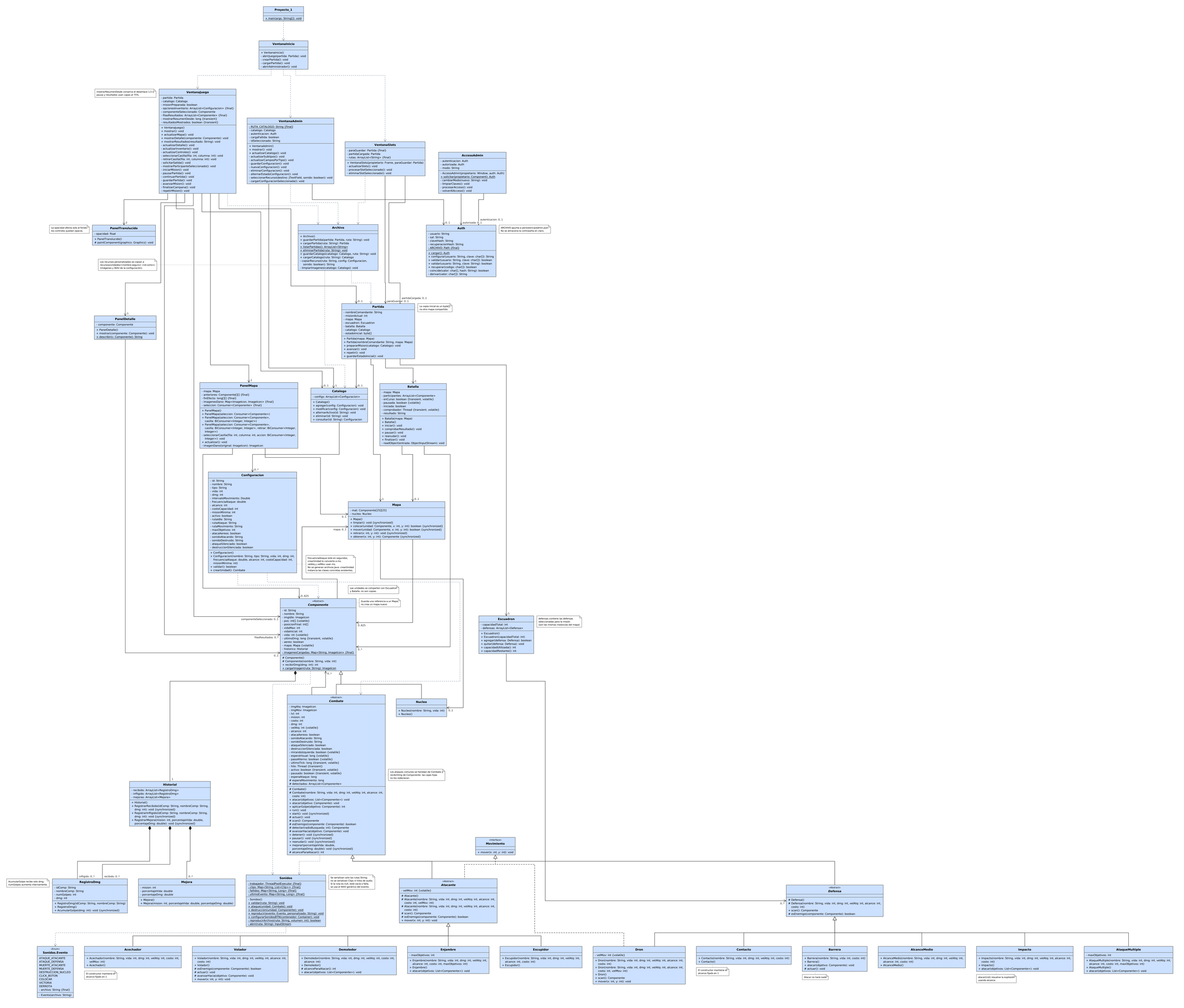
Proyecto_1
+ main(args: String[]): void
VentanaInicio
+ VentanaInicio()
- abrirJuego(partida: Partida): void
- crearPartida(): void
- cargarPartida(): void
- abrirAdministrador(): void
mostrarResumenDesde conserva el deserlaize 1.5 s
pausa y resultados usan capas al 75%.
VentanaJuego
- partida: Partida
- catalogo: Catalogo
- misionPreparada: boolean
- opcionesInventario: ArrayList<Configuracion> {final}
- componenteSeleccionado: Componente
- filasResultados: ArrayList<Componente> {final}
- mostrarResumenDesde: long {transient}
- resultadosMostrados: boolean {transient}
+ VentanaJuego()
+ mostrar(): void
+ actualizarMapa(): void
+ mostrarDetalle(componente: Componente): void
+ mostrarResultados(resultado: String): void
- actualizarDetalle(): void
- actualizarInventario(): void
- actualizarControles(): void
- seleccionarCasilla(fila: int, columna: int): void
- retirarCasilla(fila: int, columna: int): void
- solicitarSalida(): void
- mostrarParte(parteSeleccionado(): void
- iniciarMision(): void
- pausarPartida(): void
- continuarPartida(): void
- guardarPartida(): void
- avanzarMision(): void
- finalizarCampana(): void
- repetirMision(): void
VentanaAdmin
- RUTA_CATALOGO: String {final}
- catalogo: Catalogo
- autenticacion: Auth
- cargaFallida: boolean
- idSeleccionado: String
+ VentanaAdmin()
+ mostrar(): void
+ actualizarCatalogo(): void
- actualizarSubtipos(): void
- actualizarCamposPorTipo(): void
- guardarConfiguracion(): void
- nuevaConfiguracion(): void
- eliminarConfiguracion(): void
- alternarEstadoConfiguracion(): void
- seleccionarRecurso(destino: JTextField, sonido: boolean): void
- cargarConfiguracionSeleccionada(): void
VentanaSlots
- paraGuardar: Partida {final}
- partidaCargada: Partida
- rutas: ArrayList<String> {final}
+ VentanaSlots(propietario: Frame, paraGuardar: Partida)
- actualizarSlots(): void
- procesarSlotSeleccionado(): void
- eliminarSlotSeleccionado(): void
AccesoAdmin
- autenticacion: Auth
- autorizada: Auth
- modo: String
- AccesoAdmin(propietario: Window, auth: Auth)
+ solicitar(propietario: Component): Auth
- cambiarModo(nuevo: String): void
- limpiarClaves(): void
- procesarAcceso(): void
- volverAlAcceso(): void
La opacidad afecta solo al fondo;
los controles quedan opacos.
PanelTranslucido
- opacidad: float
+ PanelTranslucido()
# paintComponent(graphics: Graphics): void
Archivo
+ Archivo()
+ guardarPartida(partida: Partida, ruta: String): void
+ cargarPartida(ruta: String): Partida
+ listarPartidas(): ArrayList<String>
+ eliminarPartida(ruta: String): void
+ guardarCatalogo(catalogo: Catalogo, ruta: String): void
+ cargarCatalogo(ruta: String): Catalogo
- copiarRecurso(ruta: String, config: Configuracion, sonido: boolean): String
- limpiarImagenes(catalogo: Catalogo): void
Auth
- usuario: String
- sal: String
- claveHash: String
- recuperacionHash: String
- ARCHIVO: Path {final}
+ cargar(): Auth
+ configurar(usuario: String, clave: char[]): String
+ validar(usuario: String, clave: char[]): boolean
+ validar(usuario: String, clave: String): boolean
+ recuperar(codigo: char[]): boolean
- coincide(valor: char[], hash: String): boolean
- derivar(valor: char[]): String
ARCHIVO apunta a persistencia/admin.json.
No se almacena la contraseña en claro.
Los recursos personalizados se copian a
recursos/unidades/<nombre-seguro>-<id-corto>/
(imágenes y WAV de la configuración).
PanelDetalle
- componente: Componente
+ PanelDetalle()
+ mostrar(componente: Componente): void
+ describirIc: Componente): String
Partida
- nombreComandante: String
- misionActual: int
- mapa: Mapa
- escuadron: Escuadron
- batalla: Batalla
- catalogo: Catalogo
- estadoInicial: byte[]
+ Partida(mapa: Mapa)
+ Partida(nombreComandante: String, mapa: Mapa)
+ prepararMision(catalogo: Catalogo): void
+ avanzar(): void
+ repetir(): void
+ guardarEstadoInicial(): void
La copia inicial es un byte[]
no otro mapa compartido.
partidaCargada: 0..1
paraGuardar: 0..1
PanelMapa
- mapa: Mapa
- anteriores: Componente[][] {final}
- fireFecto: long[][] {final}
- imagenesDano: Map<ImageIcon, ImageIcon> {final}
- seleccion: Consumer<Componente> {final}
+ PanelMapa()
+ PanelMapa(seleccion: Consumer<Componente>)
+ PanelMapa(seleccion: Consumer<Componente>, casilla: BiConsumer<Integer, Integer>)
+ PanelMapa(seleccion: Consumer<Componente>, casilla: BiConsumer<Integer, Integer>, retirar: BiConsumer<Integer, Integer>)
- seleccionarCasilla(fila: int, columna: int, accion: BiConsumer<Integer, Integer>): void
+ actualizar(): void
- imagenDano(original: ImageIcon): ImageIcon
Catalogo
- configs: ArrayList<Configuracion>
+ Catalogo()
+ agregar(config: Configuracion): void
+ modificar(config: Configuracion): void
+ alternarActivo(id: String): void
+ eliminar(id: String): void
+ consultar(id: String): Configuracion
Batalla
- mapa: Mapa
- participantes: ArrayList<Componente>
- enCurso: boolean {transient, volatile}
- pausada: boolean {volatile}
- iniciada: boolean
- comprobador: Thread {transient, volatile}
- resultado: String
+ Batalla(mapa: Mapa)
+ Batalla()
+ iniciar(): void
+ comprobarResultado(): void
+ pausar(): void
+ reanudar(): void
+ finalizar(): void
- readObjectEntrada: ObjectInputStream): void
Configuracion
- id: String
- nombre: String
- tipo: String
- vida: int
- dmg: int
- intervaloMovimiento: Double
- frecuenciaAtaque: double
- alcance: int
- costoCapacidad: int
- misionMinima: int
- activo: boolean
- rutaIdle: String
- rutaAtaque: String
- rutaMovimiento: String
- maxObjetivos: int
- atacaAereos: boolean
- sonidoAtacando: String
- sonidoDestruido: String
- ataqueSilenciado: boolean
- destruccionSilenciada: boolean
+ Configuracion()
+ Configuracion(nombre: String, tipo: String, vida: int, dmg: int, frecuenciaAtaque: double, alcance: int, costoCapacidad: int, misionMinima: int)
+ validar(): boolean
+ crearUnidad(): Combate
frecuenciaAtaque está en segundos;
crearUnidad lo convierte a ms.
velMov y velMov usan ms.
No se generan archivos Java: crearUnidad instancia las clases concretas existentes.
Mapa
- mat: Componente[25][25]
- nucleo: Nucleo
+ Mapa()
+ limpiar(): void {synchronized}
+ colocar(unidad: Componente, x: int, y: int): boolean {synchronized}
+ mover(unidad: Componente, x: int, y: int): boolean {synchronized}
+ retirar(x: int, y: int): void {synchronized}
+ obtener(x: int, y: int): Componente {synchronized}
Las unidades se comparten con Escuadron
y Batalla: no son copias.
Guarda una referencia a un Mapa,
no crea un mapa nuevo
«Abstract»
Componente
- id: String
- nombre: String
- imgIdle: ImageIcon
- pos: int[] {volatile}
- posicionFinal: int[]
- vidaMax: int
- vidaInicial: int
- vida: int {volatile}
- ultimoDmg: long {transient, volatile}
- aereo: boolean
- mapa: Mapa {volatile}
- historico: Historial
- imagenesCargadas: Map<String, ImageIcon> {final}
Componente()
Componente(nombre: String, vida: int)
+ recibirDmg(dmg: int): int
+ cargarImagen(ruta: String): ImageIcon
componenteSeleccionado: 0..1
filasResultados: 0..*
Escuadron
- capacidadTotal: int
- defensas: ArrayList<Defensa>
+ Escuadron()
+ Escuadron(capacidadTotal: int)
+ agregar(defensa: Defensa): boolean
+ quitar(defensa: Defensa): void
+ capacidadUtilizada(): int
+ capacidadRestante(): int
defensas contiene las defensas
seleccionadas para la misión
(son las mismas instancias del mapa)
«Abstract»
Combate
- imgAtq: ImageIcon
- imgMov: ImageIcon
- lvl: int
- mision: int
- costo: int
- dmg: int
- velAtq: int {volatile}
- alcance: int
- ataca Aereos: boolean
- sonidoAtacando: String
- sonidoDestruido: String
- ataqueSilenciado: boolean
- destruccionSilenciada: boolean
- mirandoIzquierda: boolean {volatile}
- esperaVisual: long {volatile}
- pasoAlterno: boolean {volatile}
- ultimoTick: long {transient, volatile}
- hilo: Thread {transient}
- activo: boolean {transient, volatile}
- pausado: boolean {transient, volatile}
- esperaAtaque: long
esperaMovimiento: long
detectados: ArrayList<Componente>
Combate()
Combate(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
+ atacar(objetivos: List<Componente>): void
+ atacar(objetivo: Componente): void
+ aplicarGolpe(objetivo: Componente): int
+ run(): void
+ start(): void {synchronized}
# actuar(): void
# scan(): Componente
# esEnemigo(componente: Componente): boolean
# detectar(radioBusqueda: int): Componente
# avanzarHacia(objetivo: Componente): void
+ detener(): void {synchronized}
+ pausar(): void {synchronized}
+ reanudar(): void {synchronized}
+ mejorar(porcentajeVida: double, porcentajeDmg: double): void {synchronized}
# alcanceParaAtacar(): int
Los ataques comunes se heredan de Combate y
recibirDmg de Componente: las cajas hijas
no los redeclaran.
Nucleo
+ Nucleo(nombre: String, vida: int)
+ Nucleo()
Historial
- recibido: ArrayList<RegistroDmg>
- infligido: ArrayList<RegistroDmg>
- mejoras: ArrayList<Mejora>
+ Historial()
+ RegistrarRecibido(idComp: String, nombreComp: String, dmg: int): void {synchronized}
+ RegistrarInfligido(idComp: String, nombreComp: String, dmg: int): void {synchronized}
+ RegistrarMejora(mision: int, porcentajeVida: double, porcentajeDmg: double): void {synchronized}
«Interface»
Movimiento
+ mover(x: int, y: int): void
infligido: 0..*
recibido: 0..*
AcumularGolpe recibe solo dmg;
numGolpes aumenta internamente.
RegistroDmg
- idComp: String
- nombreComp: String
- numGolpes: int
- dmg: int
+ RegistroDmg(idComp: String, nombreComp: String)
+ RegistroDmg()
+ AcumularGolpe(dmg: int): void {synchronized}
Mejora
- mision: int
- porcentajeVida: double
- porcentajeDmg: double
+ Mejora()
+ Mejora(mision: int, porcentajeVida: double, porcentajeDmg: double)
Sonidos
- trabajador: ThreadPoolExecutor {final}
- clips: Map<String, List<Clip>> {final}
- fallidos: Map<String, Long> {final}
- ultimoEvento: Map<String, Long> {final}
- Sonidos()
+ validar(ruta: String): void
+ ataque(unidad: Combate): void
+ destruccion(unidad: Componente): void
+ reproducir(evento: Evento, personalizado: String): void
+ configurarSonidosBTN(contenedor: Container): void
- reproducirArchivo(ruta: String, volumen: int): boolean
- abrir(ruta: String): InputStream
Se serializan solo las rutas String;
no se serializan Clips ni hilos de audio.
Si la ruta es null, está vacía o falla,
se usa el WAV genérico del evento.
«Abstract»
Atacante
- velMov: int {volatile}
Atacante()
Atacante(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int, velMov: int)
Atacante(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
scan(): Componente
esEnemigo(componente: Componente): boolean
+ mover(x: int, y: int): void
«Abstract»
Defensa
Defensa()
Defensa(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
+ scan(): Componente
# esEnemigo(componente: Componente): boolean
«Enum»
Sonidos.Evento
ATAQUE_ATACANTE
ATAQUE_DEFENSA
MUERTE_ATACANTE
MUERTE_DEFENSA
DESTRUCCION_NUCLEO
CLICK_BOTON
COLOCAR
VICTORIA
DERROTA
- archivo: String {final}
- Evento(archivo: String)
Acechador
+ Acechador(nombre: String, vida: int, dmg: int, velAtq: int, costo: int, velMov: int)
+ Acechador()
El constructor mantiene el
alcance fijado en 1
Volador
+ Volador(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
+ Volador()
# esEnemigo(componente: Componente): boolean
# actuar(): void
# avanzarHacia(objetivo: Componente): void
+ mover(x: int, y: int): void
Demoledor
+ Demoledor(nombre: String, vida: int, dmg: int, velAtq: int, costo: int, alcance: int)
+ Demoledor()
# alcanceParaAtacar(): int
+ atacar(objetivos: List<Componente>): void
Enjambre
- maxObjetivos: int
+ Enjambre(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int, maxObjetivos: int)
+ Enjambre()
+ atacar(objetivos: List<Componente>): void
Escupidor
+ Escupidor(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
+ Escupidor()
Dron
- velMov: int {volatile}
+ Dron(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
+ Dron(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int, velMov: int)
+ Dron()
+ scan(): Componente
+ mover(x: int, y: int): void
Contacto
+ Contacto(nombre: String, vida: int, dmg: int, velAtq: int, costo: int)
+ Contacto()
El constructor mantiene el
alcance fijado en 1
Barrera
+ Barrera(nombre: String, vida: int, costo: int)
+ Barrera()
+ atacar(objetivo: Componente): void
# actuar(): void
Atacar no hará nada
AlcanceMedio
+ AlcanceMedio(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
+ AlcanceMedio()
Impacto
+ Impacto(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int)
+ Impacto()
+ atacar(objetivos: List<Componente>): void
atacarLista resuelve la explosión
usando alcance
AtaqueMultiple
- maxObjetivos: int
+ AtaqueMultiple(nombre: String, vida: int, dmg: int, velAtq: int, alcance: int, costo: int, maxObjetivos: int)
+ AtaqueMultiple()
+ atacar(objetivos: List<Componente>): void
Herencia
Realización de interfaz
Composición
Asociación
Dependencia
Nota
Asociaciones: la multiplicidad (y el rol, cuando hay dos o es relevante) se rotula en el extremo destino. Cursiva = abstracto; subrayado = estático.